



```
python -m software_engineer_playbook
```

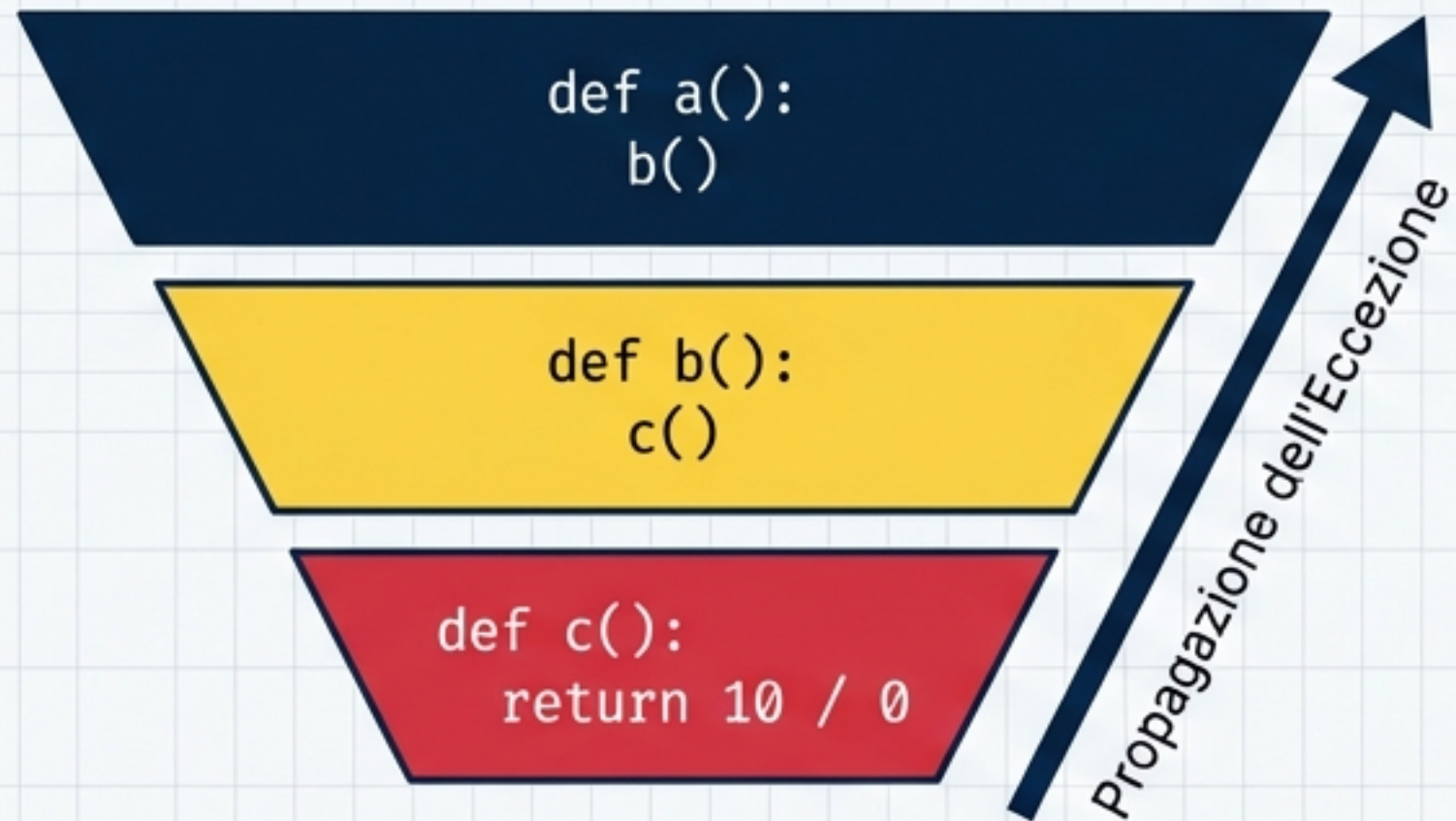
Il Blueprint del Codice Production-Ready: Dal Debugging all'Architettura Tipizzata.

[Ispezione]

[Architettura]

[Ottimizzazione]

L'Anatomia di un Bug: Decodificare lo Stack Trace



TERMINAL OUTPUT

ZeroDivisionError: division by zero

File 'app.py', line 12, in c ← L'errore nasce qui
File 'app.py', line 8, in b ← Si propaga qui
File 'app.py', line 4, in a ← Uccide il programma

Riprodurre

Isolare

Formulare
Ipotesi

Testare il
Fix

La Matrice Diagnostica: Scegliere l'Arma Giusta



Ispezione Manuale

```
print(f'totale prima: {tot}')
```

Veloce ma rudimentale. Rischioso in sistemi complessi, sporca il codice.



Difesa Attiva

```
assert b != 0, 'Div zero'
```

Ottimo per bloccare stati invalidi in sviluppo. Genera AssertionError.



Debugger Interattivo

```
if i == 5:  
    breakpoint()
```

Sospende l'esecuzione. Comandi chiave: n (next), s (step), p (print).



Tracciamento in Produzione

```
logging.error('Bug di calcolo')
```

Irrinunciabile in produzione. Supporta livelli di gravità e output su file.

Unit Testing: Il Pattern Arrange-Act-Assert



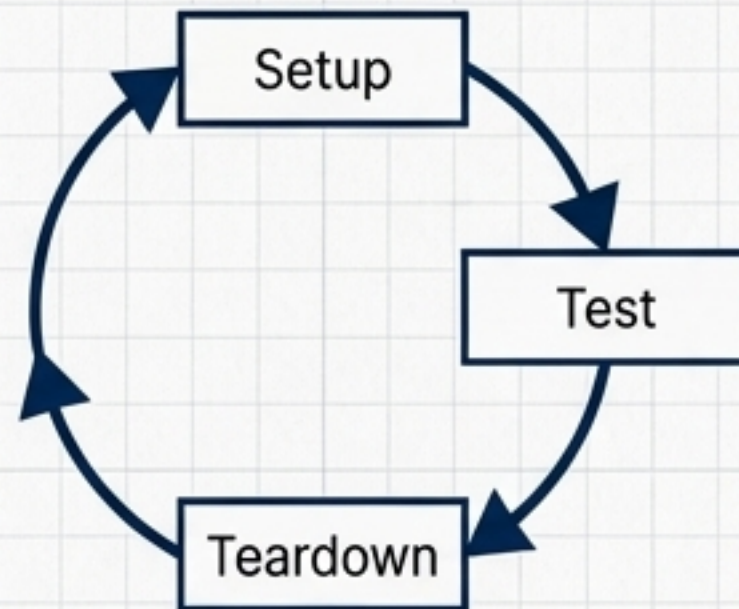
test_logic.py

```
def test_is_even():  
    # Arrange  
    input_val = 4  
    # Act  
    result = is_even(input_val)  
    # Assert  
    assert result is True
```

1 passed in 0.01s

Scalare la Qualità: Fixture e Parametrizzazione in Pytest

Il Contesto (Fixtures)



Fira Card

```
@pytest.fixture
def db_connection():
    db = connect()
    yield db
    db.close()
```

Crea dati o risorse condivise. Il comando 'yield' separa in modo pulito l'inizializzazione dalla pulizia.

La Moltiplicazione (Parametrize)

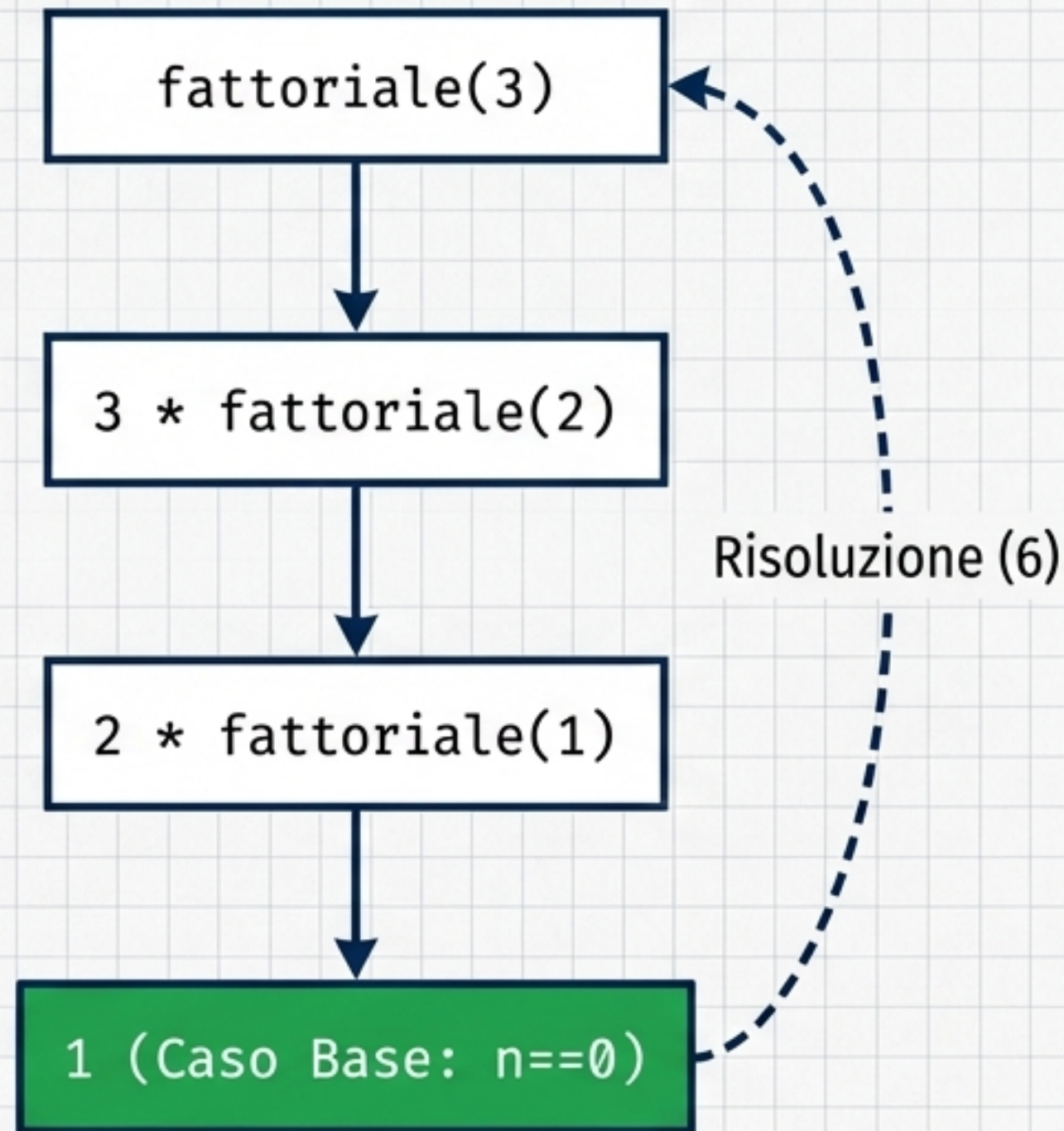


Fira Card

```
@pytest.mark.parametrize('a,b,atteso', [
    (1,2,3),
    (2,3,5),
    (10,5,15)
])
def test_somma(a, b, atteso):
    assert somma(a, b) == atteso
```

Esegue la stessa logica su un array di input. Distrugge la duplicazione e copre gli edge cases in una sola riga.

Ricorsione: Suddividere la Complessità Logica



Code Card

```
def fattoriale(n):
```

```
    if n == 0:
```

```
        return 1
```

```
    return n * fattoriale(n - 1)
```

Condizione di terminazione
(Evita lo Stack Overflow)

Chiamata a sé stessa
con input ridotto

Ricorsione	Iterazione (for/while)
Estremamente elegante per alberi/grafi	Più verbosa, meno matematica
Consuma memoria (Stack Python limitato)	Infinitamente più efficiente e sicura

Object-Oriented Programming: Dal Blueprint all'Istanza

Code Card

```
class ContoBancario:  
    def __init__(self, saldo):  
        self._saldo = saldo 🔒 — Incapsulamento:  
                                Dati protetti.  
  
    def deposita(self, importo):  
        self._saldo += importo
```

↑
'self' è il ponte vitale.
Rappresenta l'istanza corrente
che esegue l'azione.

Istanza A

```
ContoBancario(100)  
Stato: _saldo = 100
```

Istanza B

```
ContoBancario(5000)  
Stato: _saldo = 5000
```

Istanza C

```
ContoBancario(0)  
Stato: _saldo = 0
```

La Classe definisce la struttura astratta.
Gli Oggetti contengono lo stato concreto.

La Quarta Dimensione: Padroneggiare il modulo datetime



tzinfo

Agisce come una lente: altera la visualizzazione dell'ora senza cambiare il momento assoluto.

`timedelta(days=30)`

Oggi (A)

Un ponte vettoriale che quantifica una durata.

Scadenza (B)

Code Card

Da Stringa a Oggetto:

```
dt = datetime.strptime('20-03-2025', '%d-%m-%Y')
```

Da Oggetto a Stringa:

```
testo = now.strftime('%d/%m/%Y %H:%M')
```

Il Tridente dei Metodi: Istanza, Classe e Statici

Metodi di Istanza (Default)



Bersaglio

Singolo Oggetto

Code Card

```
def preleva(self, importo):
```

Modifica o legge lo stato univoco dell'istanza. Il parametro 'self' è obbligatorio.

Metodi di Classe



Bersaglio

Lo Stato Globale

Code Card

```
@classmethod  
def incrementa(cls):
```

Agisce sull'intera classe. Ottimo per creare contatori condivisi o design pattern Factory.

Metodi Statici



Bersaglio

Indipendente

Code Card

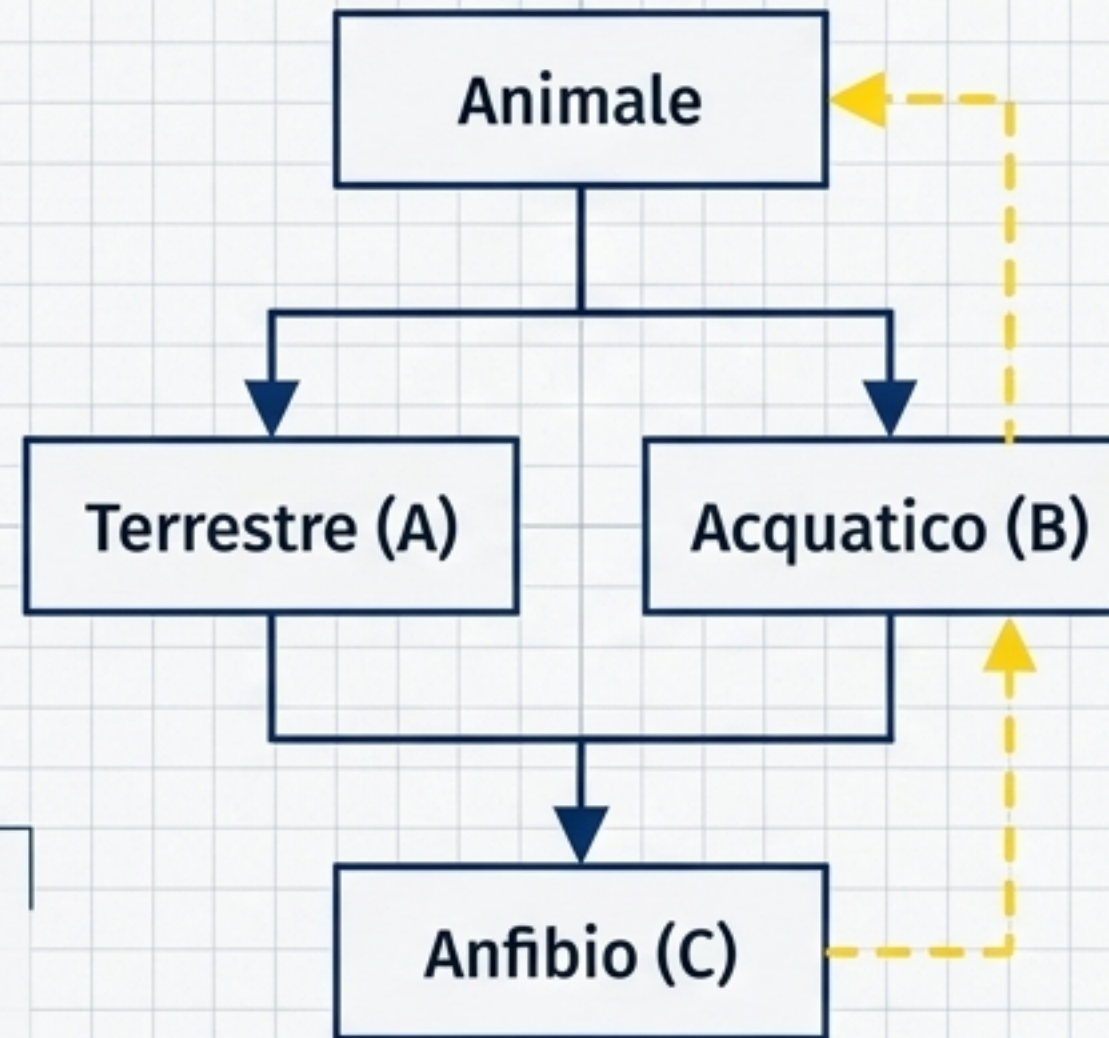
```
@staticmethod  
def quadrato(x):
```

Logica pura incapsulata per ordine. Non riceve né 'self' né 'cls'.

 **Special Ops**

Dunder Methods: '`__str__`', '`__init__`'.
Definiscono comportamenti built-in.

Ereditarietà e Method Resolution Order (MRO)



La Chiamata Magica: super()

Innesca l'MRO. Risale dinamicamente al genitore corretto senza chiamarlo per nome, garantendo robustezza strutturale.

```
super().__init__(nome)
```

Code Card

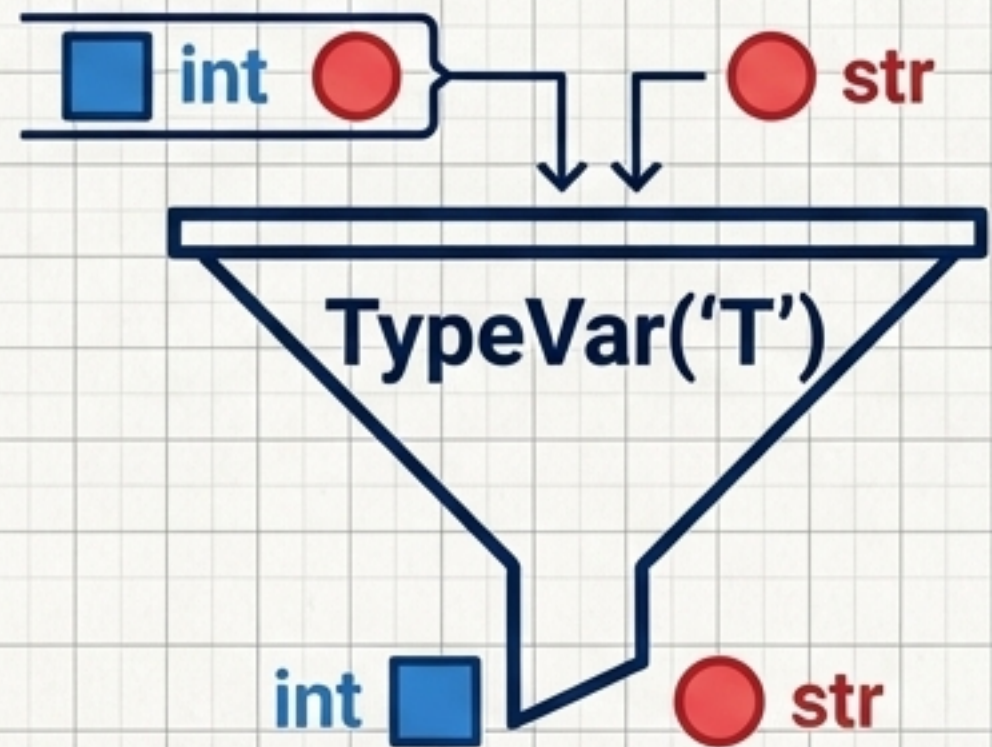
```
print(Anfibio.__mro__)

(<class 'Anfibio'>, <class 'Terrestre'>, <class 'Acquatico'>, <class 'Animale'>, <class 'object'>)
```

Riutilizzo e Override

Una classe derivata eredita tutto, ma può sovrascrivere i metodi per alterarli.

Generic Programming: Tipizzazione Dinamica Controllata



Un Generic Type non blocca la funzione su un singolo tipo, ma garantisce matematicamente che il tipo in ingresso sia coerente con quello in uscita.

Debole / Pericoloso

```
def first(lista: list) -> Any:  
    return lista[0]
```

Si perde completamente l'informazione sul tipo. L'IDE è cieco.

Ingegnere / Sicuro

```
T = TypeVar('T')  
def first(lista: list[T]) -> T:  
    return lista[0]
```

Perfetto. Il type checker sa esattamente che se inserisci list[int], otterrai un int.

Comprehensions: L'Eleganza della Trasformazione Dati

Il Passato: Ciclo For Tradizionale



Code Card

```
quadrati = []  
for x in numeri:  
    if x % 2 == 0:  
        quadrati.append(x**2)
```

Macchinoso, verboso e con un alto carico visivo.

Il Futuro: List Comprehension



Code Card

```
quadrati = [x**2 for x in numeri if x % 2 == 0]
```

[Espressione (x**2)] [for item in iterable] [if Condizione]

Una singola riga di codice brillante e performante.

Estensioni Potenti

Dict Comprehension:

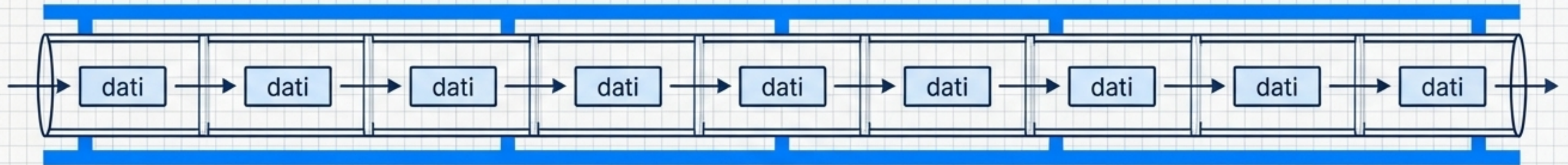
```
{k: v*0.9 for k, v in prezzi.items()}
```

Set Comprehension:

```
{x for x in lista}
```

Type Hinting: Sicurezza Statica, Anima Dinamica

Il Type Hinting non rallenta il runtime di Python. Agisce come un guardrail per l'IDE e per i linter statici (mypy).



Strutture Complesse

```
dati: dict[str, list[float]]
```

Tipizzazione profonda per API e JSON.

Assenza Esplicita

```
def trova() -> Optional[str]:
```

Equivalente a 'str | None' (Python 3.10+). Segnala esplicitamente che il valore può mancare.

Molteplicità

```
def processa(x: Union[int, str]):
```

Accetta tipi multipli controllati.



ATTENZIONE AD 'Any'. Usare 'Any' significa spegnere il type checker. È la sconfitta del type hinting.

Sintesi: Il Codice Production-Ready

Code Card

```
from datetime import datetime, timedelta
from typing import TypeVar
```

```
T = TypeVar('T')
```

```
class GestoreDate(StrumentoBase):
```

```
    def __init__(self, data_inizio: datetime):
        super().__init__()
        self._oggi = data_inizio
```

```
    def proiezioni_future(self, giorni: int) -> list[datetime]:
        return [self._oggi + timedelta(days=x) for x in range(giorni)]
```

pytest: 14 passed

OOP &
Ereditarietà

Costruttore
e MRO

Type Hinting
Rigoroso

Manipolazione
Temporale

List
Comprehension
Elegante

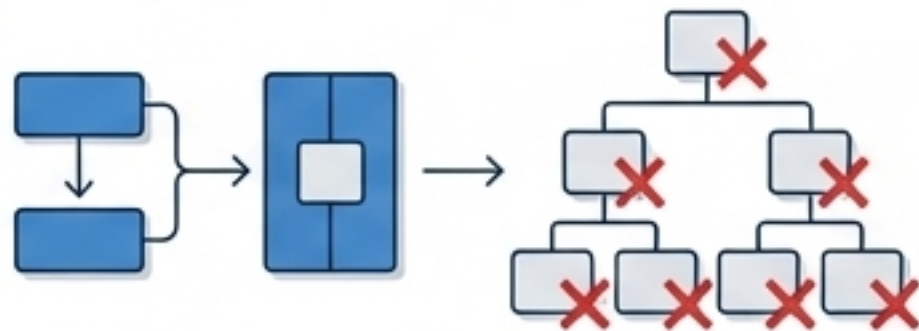
Nessun concetto vive isolato. Questa è l'intersezione elegante dell'Ingegneria del Software.

L'Ethos del Software Engineer

1. Composizione > Ereditarietà

Non creare gerarchie OOP profonde se non strettamente necessarie.

Privilegia l'assemblaggio di componenti semplici e modulari.



2. Prevenzione > Correzione

Sostituisci la speranza e le infinite `print()` con **Type Hint** rigorosi e suite di **Unit Test** automatizzati in **Pytest**.

```
def process(data: list[int]) -> bool:  
    ... = 0  
    return data
```

⚡ **pytest:**
PASSED

```
print()...  
print()...  
print()...  
print()...  
print()...
```

⊘ **ERROR**

3. Leggibilità > Compressione

Sfrutta le **Comprehensions**, ma fermati prima che diventino indecifrabili.

Il codice è letto dagli umani molto più spesso di quanto sia eseguito dalle macchine.

`[x*2 for x in range(10)]`



?

```
[x*2 for x in range(10) {  
    for e.x in range(  
        for ex in range(path),  
        [x*2 += comitefang(ax[i])]]  
}]
```

✗

Reference



Documentazione Ufficiale e PEP
(Python Enhancement Proposals)



Fluent Python (L. Ramalho)

Il viaggio verso la maestria prosegue qui.